

Short Memo: How My Simulator Mimics a Real Exchange

Executive Summary

The developed Market Simulation System is designed to replicate the complex interactions of a modern electronic communication network (ECN). By combining a **Price-Time Priority matching engine** with **heterogeneous trading agents**, the simulator reproduces "stylized facts" of financial markets, such as mean-reversion of spreads, volatility clustering, and the formation of a Limit Order Book (LOB).

I. The Core Matching Engine: Price-Time Priority

At the heart of any exchange is the matching algorithm. As shown in the **Sequence Diagram** and implemented in the LimitOrderBook class, the system enforces the two primary rules of price-time priority:

- **Price Priority:** Buy orders (Bids) with higher prices and Sell orders (Asks) with lower prices are executed first. The code achieves this using **Max-Heaps** for Bids and **Min-Heaps** for Asks, allowing O(1) access to the best available prices.
- **Time Priority:** When prices are equal, the order that entered the system first is filled first. This is managed by the order_sequence and the natural ordering of the heaps.

II. Order Lifecycle and Validation

A real exchange does not just "accept" orders; it validates and tracks them through a specific state machine.

- **Validation Phase:** As illustrated in the **UML Activity Diagram**, every order undergoes a "Syntax Check," "Price Validation," and a "Trader Credit Check" before reaching the engine.
- **State Transitions:** The **State Diagram** details the lifecycle from NEW to PENDING, then to QUEUED. An order only reaches FILLED or PARTIALLY_FILLED after successful matching logic, mirroring the rigorous accounting of real-world clearings.
- **Advanced Order Types:** Beyond simple LIMIT and MARKET orders, the implementation includes **IOC (Immediate or Cancel)** and **FOK (Fill or Kill)** logic, allowing for complex execution strategies used by institutional HFTs.

III. Agent Heterogeneity and Market Microstructure

Markets are not monolithic; they are composed of participants with conflicting goals. Our simulator mimics this through three distinct agent classes:

1. **Liquidity Providers (Market Makers):** Utilizing the Avellaneda-Stoikov model, these agents calculate a reservation price based on inventory risk:

$$r(s, t, q) = s - q\gamma\sigma^2(T - t)$$

This mimics how real market makers adjust their quotes to remain inventory-neutral.

2. **Noise Traders (Zero Intelligence):** These provide the "background radiation" of the market, ensuring there is always some level of random liquidity and price discovery, as per Gode & Sunder (1993).
3. **Liquidity Consumers (Momentum Traders):** These agents react to trends, creating the feedback loops that lead to price breakouts and volatility, common in real-world technical trading.

IV. Physical Architecture and Concurrency

While the core logic is mathematical, the **Deployment Diagram** and **Thread Communication Component Diagram** highlight the system's scalability:

- **Distributed Matching:** The system is designed to run on a **Compute Cluster** where multiple Matching Engines handle different symbols, mimicking the partitioned architecture of NASDAQ or NYSE.
- **Concurrency Primitives:** To prevent "race conditions" where two traders try to buy the same share simultaneously, the system uses **Mutexes, Semaphores, and Read-Write Locks** to ensure the integrity of the Shared Database and Order Book.
- **Low-Latency Ingress:** The **Thread 1: Order Ingress** component separates order reception from matching, ensuring the system remains responsive even during high-volume "bursts."

V. Data Persistence and Analytics

Real exchanges provide a "tape" or a data feed.

- **Persistence:** The **System Architecture** shows a dedicated **Data Layer** containing Trade History and Order History.
- **Broadcasting:** Once a trade occurs, a "Broadcast Phase" (Activity Diagram) pushes updates to a **Visualization Dashboard** and a **Time Series Database**, providing the same transparency expected by modern regulators.